

SMART EXAM SEATING & CONFLICT DETECTION SYSTEM

Project Report

Subject	CSE2006 – Programming in Java
Project Type	Java Console-Based Application
Student	Aryan Sachin Bapat
Registration No.	25BAI10560

Submitted as part of the VITyarthi project work for Programming in Java.

1. Introduction

The Smart Exam Seating & Conflict Detection System is a Java-based console application developed to make examination seating arrangements easier and more organized. The system allows examination staff to add students and examination rooms, generate seating arrangements, and check the arrangement for possible conflicts. It can detect problems such as duplicate seat allocation and students who could not be allocated a seat. The project demonstrates important Java concepts such as Object-Oriented Programming, inheritance, encapsulation, collections, multithreading, exception handling, string processing, and file handling.

2. Problem Statement

Creating examination seating arrangements manually can be time-consuming, especially when there are many students and multiple examination rooms. Manual arrangements may result in problems such as assigning two students to the same seat, exceeding room capacity, leaving students without seats, and difficulty checking the final arrangement. The proposed system automatically allocates students to available seats and checks the generated arrangement for possible conflicts.

3. Objectives

- Manage student examination information.
- Manage examination rooms and their capacities.
- Automatically generate seating arrangements.
- Prevent duplicate seat allocation.
- Detect unallocated students.
- Check seating arrangements for conflicts.
- Generate examination seating reports.
- Export reports to a text file.
- Demonstrate practical applications of Java and Object-Oriented Programming concepts.

4. Scope of the Project

The project focuses on examination seating management in a college environment. It supports adding student information, adding examination rooms, generating seating arrangements, detecting conflicts, viewing the final arrangement, generating reports, and exporting seating information to a text file. The current version is a console-based Java application and uses Java collections to store information during program execution.

5. Target Users

Examination Department: Can use the system to generate and verify seating arrangements.

Faculty / Administrators: Can manage students, rooms, seating arrangements, and conflict checking.

6. Functional Requirements

Student Management: Add and view student examination details.

Room Management: Store examination rooms and capacities.

Seat Allocation: Assign students to available seats.

Conflict Detection: Check duplicate seats, duplicate students, and unallocated students.

Seating Arrangement: Display the final student-to-seat allocation.

Report Generation: Generate seating statistics and reports.

Report Export: Export the report to a text file.

7. Non-Functional Requirements

Usability: Simple menu-driven console interface.

Reliability: Checks allocations and handles common input errors.

Maintainability: Separate classes handle different responsibilities.

Error Handling: Exception handling is used for invalid input and runtime errors.

Performance: Collection-based allocation is suitable for a small or medium examination setup.

8. System Architecture

The application follows a modular architecture. Main.java controls user interaction. StudentManager manages students, RoomManager manages examination rooms, SeatingManager handles allocation, ConflictDetector checks the arrangement, ReportGenerator creates reports, and FileManager exports the report.

Architecture Flow:

Examination Staff → Main Application → StudentManager / RoomManager → SeatingManager → ConflictDetector → ReportGenerator → FileManager

9. System Workflow

1. Start the application.
2. Add student examination details.
3. Add examination rooms and their capacities.
4. Generate the seating arrangement.
5. Assign students to available seats.
6. Run the conflict detection process.
7. View the final seating arrangement.
8. Generate a seating report.
9. Export the report to a text file.

10. Class Design

User: Base class containing common user information.

Student: Extends User and stores student/course details.

Admin: Extends User and represents an administrator.

ExamRoom: Stores room number and seating capacity.

Seat: Represents a seat and assigned student.

SeatingArrangement: Stores and displays generated allocations.

StudentManager: Stores and validates students using ArrayList.

RoomManager: Stores and validates rooms.

SeatingManager: Generates the seating arrangement.

ConflictDetector: Checks duplicate allocations and unallocated students.

SeatingCheckThread: Runs conflict checking using Thread.

ReportGenerator: Creates seating statistics.

FileManager: Exports reports to a text file.

Main: Controls menu and application flow.

11. Java Concepts Used

Classes and Objects: Separate classes represent students, rooms, seats, arrangements, managers, and reports.

Encapsulation: Data is stored inside classes and accessed through methods.

Inheritance: Student and Admin extend User.

Method Overriding: Student and Admin override displayUser().

Polymorphism: Parent User references can represent different User subclasses.

Collections: ArrayList stores students and rooms, while HashSet helps detect duplicates.

Multithreading: SeatingCheckThread extends Thread for conflict checking.

Exception Handling: try-catch handles invalid input and runtime errors.

File I/O: FileManager uses FileWriter to export reports.

12. Seating Allocation Approach

The SeatingManager takes the list of students and examination rooms. For each room, it creates seats according to the room capacity and assigns students sequentially to available seats. If total room capacity is less than the number of students, some students remain unallocated and the conflict detector reports the issue.

Example:

AB101 – Capacity 2

Student 1 → AB101 / Seat-1

Student 2 → AB101 / Seat-2

13. Conflict Detection

The ConflictDetector checks the generated arrangement for possible problems. A HashSet stores student roll numbers to detect duplicate student allocation. Another HashSet stores room and seat identifiers to detect duplicate seat allocation. The system also compares the number of allocated students with the total number of students to identify unallocated students.

14. Multithreading

The project uses Java multithreading for the conflict-checking operation. SeatingCheckThread extends the Thread class and invokes ConflictDetector. This demonstrates how a separate thread can be used to perform a checking operation within the application.

15. Implementation

The application starts from Main.java and provides a menu for adding students, adding rooms, viewing information, generating the seating arrangement, checking conflicts, viewing the arrangement, generating a report, and exporting the report. StudentManager and RoomManager maintain input data using ArrayList. SeatingManager creates the allocation. ConflictDetector verifies the result. ReportGenerator calculates statistics and FileManager writes the report to a text file.

16. Project Structure

SmartExamSeatingSystem/

■■■ User.java

■■■ Student.java

■■■ Admin.java

■■■ ExamRoom.java

■■■ Seat.java

■■■ SeatingArrangement.java

■■■ StudentManager.java

■■■ RoomManager.java

■■■ SeatingManager.java

■■■ ConflictDetector.java

■■■ SeatingCheckThread.java

■■■ ReportGenerator.java

■■■ FileManager.java

■■■ Main.java

17. Sample Test Data

Students:

23ABC001 – Rahul – CSE
23ABC002 – Aman – CSE
23ABC003 – Rohan – CSE
23ABC004 – Karan – CSE

Rooms:

AB101 – Capacity 2
AB102 – Capacity 2

Generated Arrangement:

AB101 – Seat-1 → Rahul
AB101 – Seat-2 → Aman
AB102 – Seat-1 → Rohan
AB102 – Seat-2 → Karan

18. Testing

TC1: Add valid student → Student added.
TC2: Add room → Room added.
TC3: Generate arrangement → Students assigned to seats.
TC4: More students than capacity → Unallocated conflict detected.
TC5: Duplicate seat → Duplicate seat detected.
TC6: Duplicate student → Duplicate student detected.
TC7: View arrangement → Allocation displayed.
TC8: Generate report → Statistics displayed.
TC9: Export report → Text report created.
TC10: Invalid input → Error handled without crash.

19. Results

The implemented system successfully performs the main operations required for examination seating management. It can store student information, store room information, allocate students to available seats, detect duplicate allocations, identify unallocated students, display the seating arrangement, generate reports, and export reports to a text file.

20. Challenges Faced

- Designing a clear method for allocating students across multiple rooms.
- Preventing duplicate student and seat assignments.
- Detecting different types of seating conflicts.
- Handling invalid user input.
- Dividing the project into multiple classes with clear responsibilities.
- Implementing conflict checking using a separate Java thread.

21. Learning Outcomes

Through this project, practical knowledge was gained in Java classes and objects, inheritance, encapsulation, method overriding, polymorphism, collections, HashSet, multithreading, exception handling, string processing, file handling, modular programming, and basic algorithm design.

22. Limitations

- The application is console-based.
- Student, room, and seating data is stored in memory.
- Data is not permanently stored after the program terminates.
- The current allocation algorithm is a basic academic implementation.
- The system does not currently provide a graphical interface.

23. Future Enhancements

- MySQL database integration using JDBC for permanent storage.
- Graphical User Interface.
- Student and administrator login system.
- Advanced seating rules, such as separating students from the same course or subject.
- More optimized seating algorithms.
- Web-based interface.
- Integration with examination scheduling systems.

24. Conclusion

The Smart Exam Seating & Conflict Detection System provides a practical solution for creating and checking examination seating arrangements. It reduces manual effort and helps identify common problems such as duplicate seat allocation and unallocated students. The project successfully demonstrates important Java concepts including Object-Oriented Programming, inheritance, encapsulation, collections, multithreading, exception handling, string processing, and file handling.

25. References

1. CSE2006 – Programming in Java course syllabus.
2. VITyarthi Build Your Own Project guidelines.
3. Java programming concepts and standard Java libraries used during development.
4. GitHub documentation for project organization and version control.